SWAPWEAR

Clothing Exchange Marketplace

Professional Project Report

Domain: Full Stack Web Development Technologies: HTML, CSS, JavaScript, Node.js, Express.js, MongoDB
Project focus: Sustainable fashion and clothing reuse

# Table of Contents

1. Abstract

2. Introduction

3. Problem Statement

4. Objectives

5. Technologies Used

6. System Modules

7. System Architecture

8. Database Design

9. System Workflow

10. User Interface Screenshots

11. Advantages

12. Limitations

13. Future Scope

14. Conclusion

15. References

# 1. Abstract

SwapWear is a web-based clothing exchange marketplace designed to help users browse clothing items and initiate exchange requests. The project promotes reuse of wearable clothing, supports sustainable fashion, and aims to reduce textile waste. The frontend provides a visual interface for exploring listings, searching and filtering items, adding clothing, and reviewing swap requests. The backend is built with Node.js and Express.js, while MongoDB is used to store application data.

# 2. Introduction

Clothing is often discarded while it is still usable, contributing to textile waste. SwapWear provides a digital platform where users can discover clothing items and express interest in exchanging them. The project brings together frontend design, backend API development, and database integration in a single web application.

# 3. Problem Statement

People may have unused clothing that remains in their wardrobes, while others are looking for different clothing items. A simple online platform can help users discover available items and start an exchange process. SwapWear is designed to support this process through listings and swap requests.

# 4. Objectives

Build a user-friendly clothing exchange website.

Display clothing listings with images and product information.

Provide search and filtering options for listings.

Allow users to submit and review swap requests.

Provide pages for adding items, user dashboard, chat, and profile.

Use a backend API and database for application data.

Encourage clothing reuse and sustainable fashion.

# 5. Technologies Used

Technology | Purpose

HTML5 | Defines the structure of web pages.

CSS3 | Styles page layouts and supports responsive design.

JavaScript | Adds interactive behavior, filters, and frontend API calls.

Node.js | Runs JavaScript on the server.

Express.js | Defines backend routes and REST API endpoints.

MongoDB | Stores user, item, swap request, and message documents.

Mongoose | Provides schema-based interaction with MongoDB.

GitHub / Render | Source control and backend hosting, where configured.

# 6. System Modules

## 6.1 Home Page

Introduces SwapWear and directs users to browse clothes or get started.

## 6.2 User Authentication

Provides registration and login entry points for users.

## 6.3 Clothing Listings

Displays item images, names, categories, sizes, conditions, and location details. Search and category/size filters help users explore items.

## 6.4 Add Item

Provides a form interface for entering clothing details and selecting an item image.

## 6.5 Swap Requests

Allows a user to initiate a request for a listed item. Request records can include requester, owner, item, message, and status fields.

## 6.6 Dashboard

Displays swap request cards and their current statuses, such as pending, accepted, or rejected.

## 6.7 Chat and Profile

Dedicated pages are included in the website navigation for user communication and profile information.

# 7. System Architecture

SwapWear follows a client-server architecture. The browser-based frontend sends HTTP requests to the Express.js backend. Backend routes validate and process requests, Mongoose communicates with MongoDB, and the API returns a response for the frontend to display.

User | Frontend HTML/CSS/JS | Express.js API | Mongoose | MongoDB

Flow: User action → Frontend request → Backend route → Database operation → API response → UI update.

## 8. Database Design

MongoDB stores data as documents within collections. The project environment includes collections for items, users, messages, and swap requests. The exact schema depends on the Mongoose models.

Users: account and profile information.

Items: clothing details and ownership information.

Swap requests: item reference/details, requester, owner, status, message, and timestamps.

Messages: chat-related information.

Development note: the local MongoDB database used by the backend must match the database inspected in MongoDB Compass. A hosted database and a local database are separate unless the connection string is configured to point to the same database.

## 9. System Workflow

User opens the website.

User registers or logs in.

User browses and filters clothing listings.

User selects an item and clicks Swap Now.

Frontend sends the swap request to the backend API.

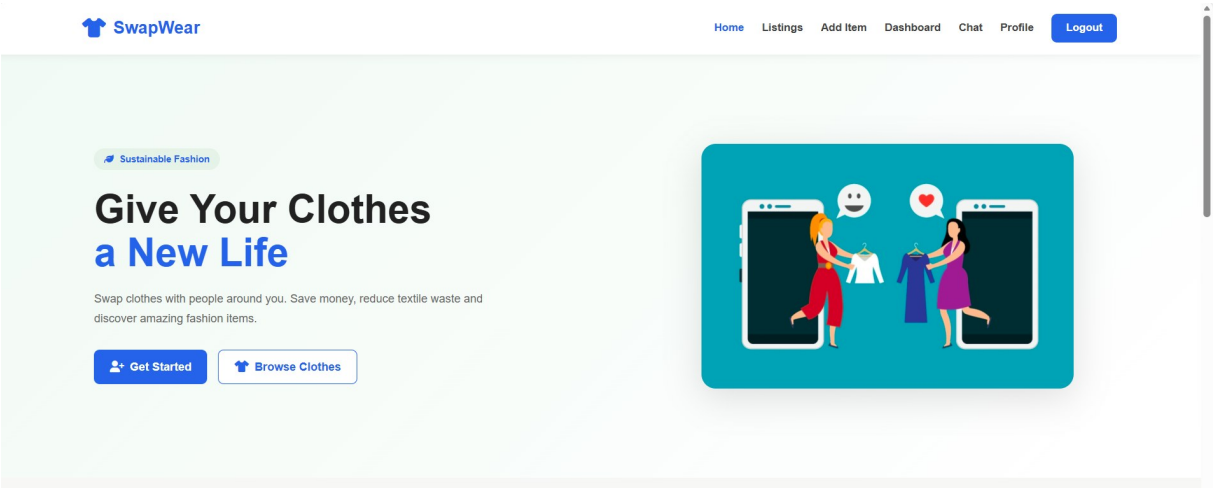
Backend validates the request and stores it in MongoDB when successful.

Dashboard retrieves request information and displays the status.

## 10. User Interface Screenshots

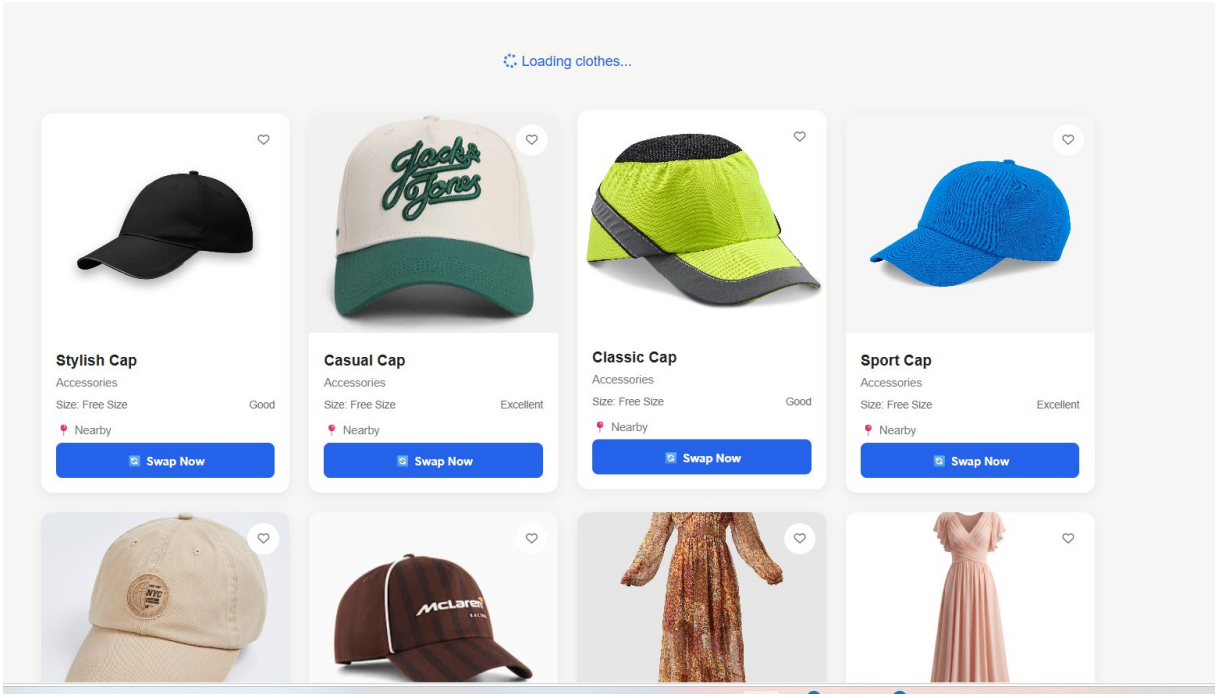
The following screenshots were supplied for this report and illustrate the main user-facing pages of the SwapWear application.

Figure 1. SwapWear Home Page



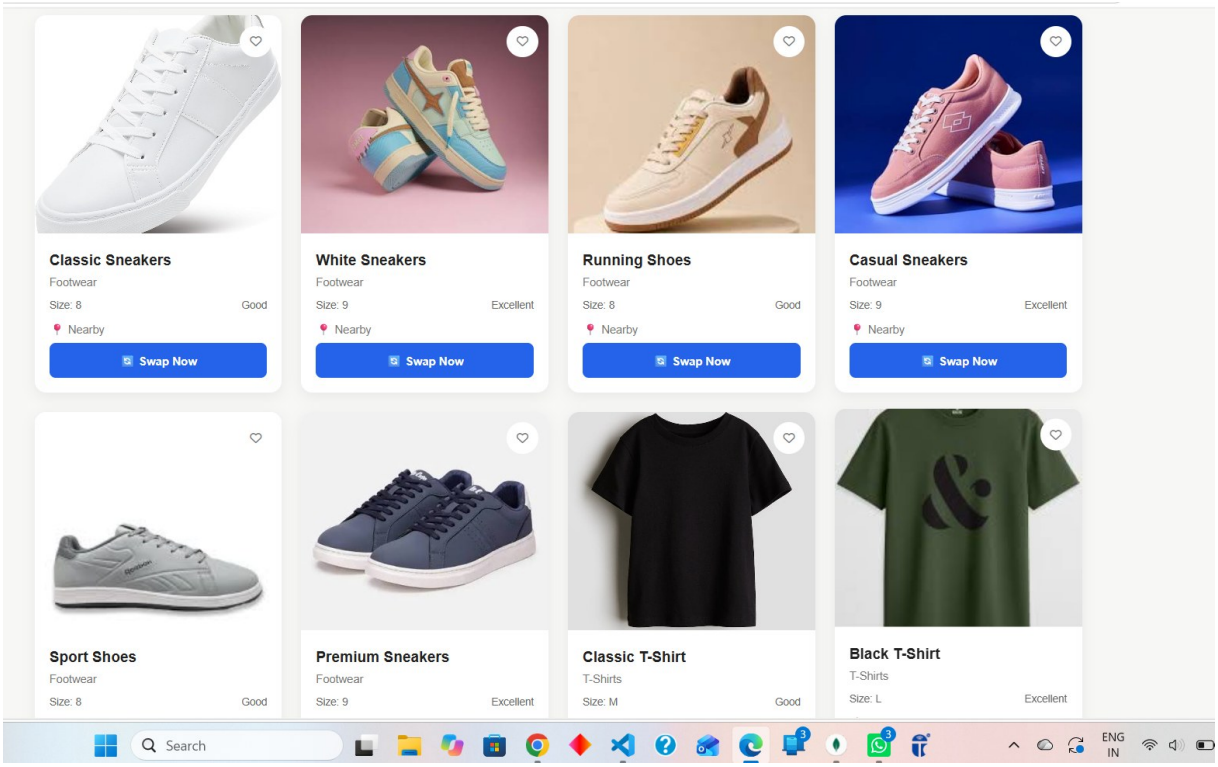
The landing page introduces the sustainable fashion concept and provides navigation to the main features.

Figure 2. Browse Listings – First View



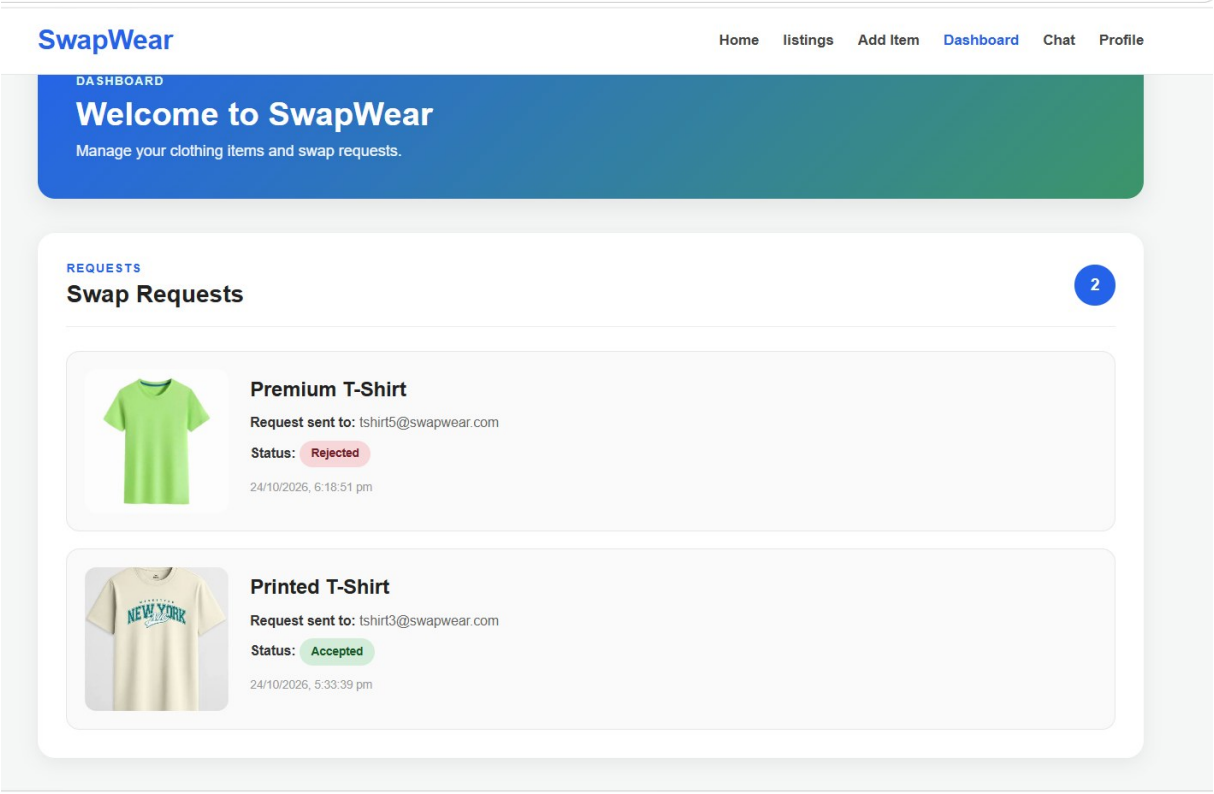

The listings interface displays clothing cards with images, product details, favourites, and Swap Now actions.

Figure 3. Browse Listings – Additional Items



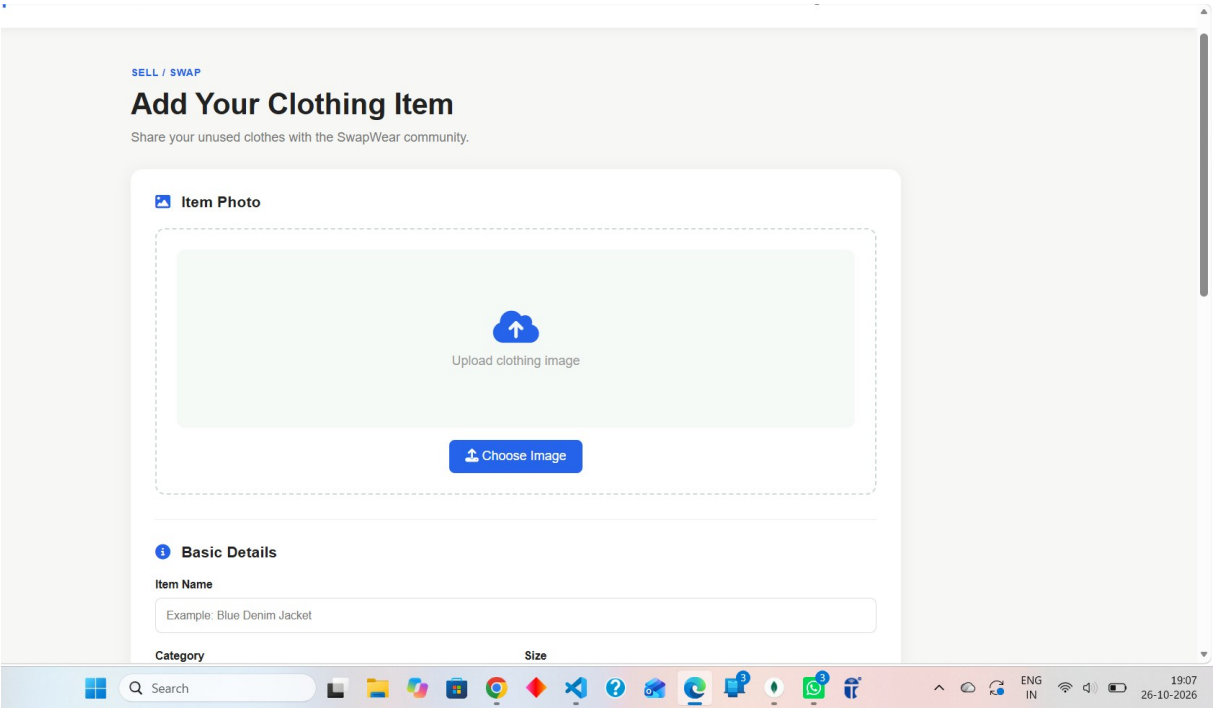

Additional listing cards demonstrate the range of footwear and clothing products.

Figure 4. User Dashboard and Swap Requests




The dashboard shows request cards with product names, recipient information, status labels, and timestamps.

Figure 5. Add Clothing Item Page




The add-item interface provides an image selection area and fields for entering item details.

## 11. Advantages

Encourages clothing reuse and sustainable fashion.

Makes clothing listings accessible through a single interface.

Search and filters help users find relevant items.

Demonstrates practical integration of frontend, backend, and database technologies.

Provides a foundation for further marketplace features.

## 12. Limitations

Successful request storage depends on correct frontend API configuration and backend/database connectivity.

Local MongoDB and MongoDB Atlas may contain different data if they are not configured to use the same database.

Dashboard information must be fetched and filtered using the correct logged-in user and request fields.

Additional testing is required to verify all flows, security controls, and error cases.

## 13. Future Scope

Real-time messaging and notifications.

Cloud image storage for uploaded clothing photos.

User ratings and reviews.

Location-based item discovery and advanced search.

Admin tools for moderation and reporting.

Improved authorization, validation, automated testing, and production deployment.

## 14. Conclusion

SwapWear is a web application concept focused on making clothing exchange easier and encouraging sustainable fashion. The project demonstrates the use of HTML, CSS, JavaScript, Node.js, Express.js, and MongoDB in a full stack application. Its main interfaces support browsing listings, adding items, and reviewing swap requests. Further integration testing and refinement of request synchronization can improve reliability and user experience.

## 15. References

MDN Web Docs — HTML, CSS, and JavaScript documentation.

Node.js Documentation — https://nodejs.org/docs/

Express.js Documentation — https://expressjs.com/

MongoDB Documentation — https://www.mongodb.com/docs/

Mongoose Documentation — https://mongoosejs.com/docs/

GitHub Documentation — https://docs.github.com/

Render Documentation — https://render.com/docs